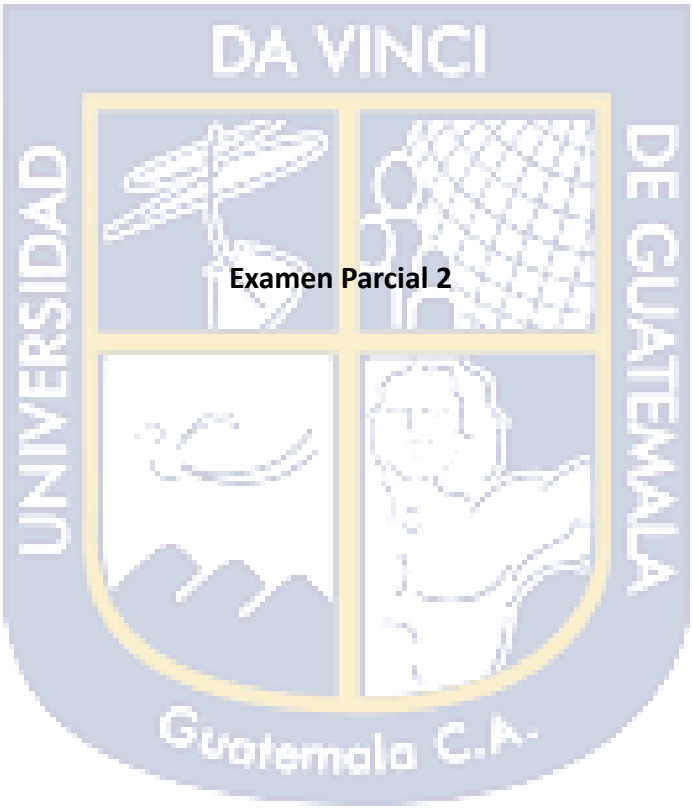
Universidad Davinci de Guatemala

Curso: Estructura de datos



Examen Parcial 2

Nombre: Alberto José Soto Zuñiga.

No. Carne: 201902254

1. Introducción

En este trabajo se desarrolla una Lista Doblemente Enlazada Circular en lenguaje Java. Esta estructura se caracteriza porque cada nodo contiene un dato y dos referencias: una al nodo siguiente y otra al nodo anterior. También, posee la propiedad de circularidad, lo que significa que el último nodo apunta nuevamente al primero y el primero apunta al último, formando un ciclo cerrado.

2. Objetivos específicos

Definir la clase **(Nodo)** con sus enlaces anterior y siguiente.

Implementar la clase (**ListaDobleCircular)** con las operaciones pedidas.

Desarrollar un menú interactivo en la clase **(Main)**.

Explicar el funcionamiento lógico de cada método.

Mostrar ejemplos de ejecución para validar el comportamiento del programa.

3. Marco teórico

Una lista doblemente enlazada circular es una estructura dinámica compuesta por nodos. Cada nodo tiene tres partes:

Un valor o dato.

Una referencia al nodo siguiente.

Una referencia al nodo anterior.

A diferencia de una lista doblemente enlazada lineal, en la versión circular:

el último nodo no apunta a null, sino al primero;

el primer nodo no apunta a null en su enlace anterior, sino al último.

Esto permite recorrer la lista desde cualquier punto y continuar indefinidamente mientras no se establezca una condición de parada.

Ventajas

Inserciones y eliminaciones eficientes en extremos.

Recorrido en ambos sentidos.

Estructura útil para ciclos y menús circulares.

Desventajas

Mayor complejidad que una lista simplemente enlazada.

Es necesario controlar bien los enlaces para no romper la circularidad.

4. **Diseño de la solución**

La solución se divide en tres clases:

Clase **Nodo**

Representa cada elemento de la lista.

Clase **ListaDobleCircular**

Contiene la lógica de todas las operaciones.

Clase **Main**

Muestra un menú repetitivo para interactuar con el usuario.

5. Código en Java

5.1 Archivo Nodo.java

```
public class Nodo {
int dato;
Nodo anterior;
Nodo siguiente;
public Nodo(int dato) {
this.dato = dato;
```

```
this.anterior = null;
this.siguiente = null;
}
}
```

Explicación

Esta clase representa la unidad básica de la estructura.

Cada nodo guarda lo siguiente:

dato: el valor entero almacenado.

anterior: referencia al nodo previo.

siguiente: referencia al nodo posterior.

El constructor recibe el valor y, al crearse, inicializa ambos enlaces en null. Más adelante, la lista se encarga de enlazar correctamente cada nodo.

5.2Archivo ListaDobleCircular.java

```
public class ListaDobleCircular {
   private Nodo head;
   public ListaDobleCircular() {
      head = null;
   }
   public boolean estaVacia() {
      return head == null;
   }
   public void insertarAlInicio(int dato) {
      Nodo nuevoNodo = new Nodo(dato);
      if (head == null) {
         nuevoNodo.siguiente = nuevoNodo;
```

```
        nuevoNodo.anterior = nuevoNodo;
        head = nuevoNodo;
    } else {
        Nodo ultimo = head.anterior;
        nuevoNodo.siguiente = head;
        nuevoNodo.anterior = ultimo;
        ultimo.siguiente = nuevoNodo;
        head.anterior = nuevoNodo;
        head = nuevoNodo;
    }
    System.out.println("Elemento insertado al inicio correctamente.");
}
public void insertarAlFinal(int dato) {
    Nodo nuevoNodo = new Nodo(dato);
    if (head == null) {
        nuevoNodo.siguiente = nuevoNodo;
        nuevoNodo.anterior = nuevoNodo;
        head = nuevoNodo;
    } else {
        Nodo ultimo = head.anterior;
        nuevoNodo.siguiente = head;
        nuevoNodo.anterior = ultimo;
        ultimo.siguiente = nuevoNodo;
        head.anterior = nuevoNodo;
    }
    System.out.println("Elemento insertado al final correctamente.");
}
public void eliminarAlInicio() {
    if (head == null) {
```

```
        System.out.println("Error: la lista está vacía.");
        return;
    }
    if (head.siguiente == head) {
        head = null;
    } else {
        Nodo ultimo = head.anterior;
        head = head.siguiente;
        head.anterior = ultimo;
        ultimo.siguiente = head;
    }
    System.out.println("Elemento eliminado al inicio correctamente.");
}
public void eliminarAlFinal() {
    if (head == null) {
        System.out.println("Error: la lista está vacía.");
        return;
    }
    if (head.siguiente == head) {
        head = null;
    } else {
        Nodo ultimo = head.anterior;
        Nodo penultimo = ultimo.anterior;
        penultimo.siguiente = head;
        head.anterior = penultimo;
    }
    System.out.println("Elemento eliminado al final correctamente.");
}
public boolean buscar(int valorBuscado) {
```

```
        if (head == null) {
            return false;
        }
    Nodo actual = head;
        do {
            if (actual.dato == valorBuscado) {
                return true;
            }
            actual = actual.siguiente;
        } while (actual != head);
        return false;
    }
    public void imprimir() {
        if (head == null) {
            System.out.println("La lista está vacía");
            return;
        }
        Nodo actual = head;
        System.out.print("Lista: ");
        do {
            System.out.print(actual.dato + " <-> ");
            actual = actual.siguiente;
        } while (actual != head);
        System.out.println("(circular -> " + head.dato + ")");
    }
}
```

UNIVERSIDAD DA VINCI DE GUATEMALA Guatemala C.A.

Explicación

Insertar al inicio

Lógica

Se crea un nodo nuevo.

Si la lista está vacía:

el nuevo nodo se apunta a sí mismo en siguiente y anterior;

ese nodo pasa a ser head.

Si la lista ya tiene elementos:

se obtiene el último nodo con head.anterior;

el nuevo nodo se enlaza antes de head;

se ajustan las referencias del último nodo y del anterior de head;

finalmente, head pasa a ser un nuevo nuevo nodo.

Por qué funciona

En una lista circular, el último nodo siempre está accesible mediante head.anterior. Eso permite insertar al inicio sin recorrer toda la lista.

Insertar al inicio

Lógica

- Se crea un nodo nuevo.

Si en la lista está vacía:

- el nuevo nodo se apunta a sí mismo en siguiente y anterior;
- ese nodo pasa a ser head.

Si en la lista ya tiene elementos:

UNIVERSIDAD DA VINCI DE GUATEMALA Guatemala C.A.

- se obtiene el último nodo con **(head.anterior)**;
- el nuevo nodo se enlaza antes de **head**;
- se ajustan las referencias del último nodo y del anterior de **head**;
- finalmente, **head** pasa a ser el nuevo nodo.

Por qué funciona

En una lista circular, el último nodo siempre está accesible mediante **head.anterior**. Eso permite insertar al inicio sin recorrer toda la lista.

Insertar al final

Lógica

Se crea un nodo nuevo.

Si la lista está vacía, se enlaza a sí mismo y se vuelve head.

Si ya existen nodos:

- se obtiene el último con **(head.anterior)**;
- el nuevo nodo se enlaza entre el último y **head**;
- **head** no cambia.

Diferencia clave con insertar al inicio

En esta operación no se modifica la cabeza de la lista.

Eliminar al inicio

Lógica

Si la lista está vacía, se muestra un error.

Si solo existe un nodo, head pasa a null.

Si hay más de un nodo:

- se obtiene el último;
- head avanza al siguiente nodo;
- se actualizan las referencias entre el nuevo head y el último.

importante

Se elimina lógicamente el primer nodo al mover la referencia head.

Eliminar al final

Lógica

Si la lista está vacía, se muestra error.

Si hay un solo nodo, la lista queda vacía.

Si hay varios:

- se identifica el último;
- luego el penúltimo;
- el penúltimo pasa a enlazarse con head;
- head.anterior pasa a ser el penúltimo.

Idea importante

No es necesario recorrer toda la lista si ya tenemos acceso directo al último mediante head.anterior.

Buscar un elemento

Lógica

- Si la lista está vacía, se retorna false.
- Se inicia desde head.
- Se usa un ciclo do-while porque en listas circulares siempre se debe revisar al menos el nodo inicial.
- Si se encuentra el valor, retorna true.
- Si vuelve a head y no encontró nada, retorna false.

Por qué se usa do-while

Porque garantiza revisar el primer nodo antes de validar la condición de salida.

Imprimir la lista

Lógica

Si está vacía, se informa al usuario.

Si tiene nodos:

se empieza desde head;

se imprimen todos los valores avanzando por siguiente;

el ciclo termina cuando se regresa a head.

Resultado

Una salida similar a:

Lista: 10 <-> 20 <-> 30 <-> (circular -> 10)

Resultado

=== PRUEBA DE LISTA DOBLEMENTE CIRCULAR ===

¿Está vacía? true

--- Insertando al inicio ---

Elemento insertado al inicio correctamente.

Elemento insertado al inicio correctamente.

Elemento insertado al inicio correctamente.

Lista: 1 <-> 5 <-> 10 <-> (circular -> 1)

--- Insertando al final ---

Elemento insertado al final correctamente.

Elemento insertado al final correctamente.

Lista: 1 <-> 5 <-> 10 <-> 20 <-> 30 <-> (circular -> 1)

--- Búsqueda ---

¿Buscar 5? true

¿Buscar 99? false

--- Eliminando al inicio ---

Elemento eliminado al inicio correctamente.

Lista: 5 <-> 10 <-> 20 <-> 30 <-> (circular -> 5)

--- Eliminando al final ---

Elemento eliminado al final correctamente.

Lista: 5 <-> 10 <-> 20 <-> (circular -> 5)

--- Insertar 100 al final ---

Elemento insertado al final correctamente.

Lista: 5 <-> 10 <-> 20 <-> 100 <-> (circular -> 5)

--- Insertar 999 al inicio ---

Elemento insertado al inicio correctamente.

Lista: 999 <-> 5 <-> 10 <-> 20 <-> 100 <-> (circular -> 999)

--- Vaciar la lista eliminando todo ---

Elemento eliminado al inicio correctamente.

Elemento eliminado al inicio correctamente.

Elemento eliminado al inicio correctamente.

Elemento eliminado al inicio correctamente.

Elemento eliminado al inicio correctamente.

La lista está vacía

--- Probando errores con lista vacía ---

Error: la lista está vacía.

Error: la lista está vacía.

¿Buscar 10? False

5.3 Archivo Main.java

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        ListaDobleCircular lista = new ListaDobleCircular();

        int opcion;

        int dato;


        do {

            System.out.println("========================================");

            System.out.println(" LISTA DOBLEMENTE ENLAZADA CIRCULAR");

            System.out.println("========================================");

            System.out.println("1. Insertar al inicio");

            System.out.println("2. Insertar al final");

            System.out.println("3. Eliminar al inicio");

            System.out.println("4. Eliminar al final");

            System.out.println("5. Buscar elemento");

            System.out.println("6. Imprimir lista");

            System.out.println("7. Salir");

            System.out.println("========================================");

            System.out.print("Seleccione una opción: ");

            opcion = sc.nextInt();

            switch (opcion) {

                case 1:
```

```
    System.out.print("Ingrese el dato a insertar al inicio: ");
    dato = sc.nextInt();
    lista.insertarAlInicio(dato);
    lista.imprimir();
    break;
  case 2:
    System.out.print("Ingrese el dato a insertar al final: ");
    dato = sc.nextInt();
    lista.insertarAlFinal(dato);
    lista.imprimir();
    break;
  case 3:
    lista.eliminarAlInicio();
    lista.imprimir();
    break;
  case 4:
    lista.eliminarAlFinal();
    lista.imprimir();
    break;
  case 5:
    System.out.print("Ingrese el dato a buscar: ");
    dato = sc.nextInt();
    if (lista.buscar(dato)) {
      System.out.println("Resultado: Elemento encontrado.");
    } else {
      System.out.println("Resultado: Elemento no encontrado.");
```

```
            }
            break;
        case 6:
            lista.imprimir();
            break;
        case 7:
            System.out.println("Saliendo del programa...");
            break;
        default:
            System.out.println("Opción no válida. Intente nuevamente.");
        }
        System.out.println();
    } while (opcion != 7);

    sc.close();
  }
}
```

Explicación

La clase **Main** controla la interacción con el usuario.

Funcionamiento

- Se crea un objeto Scanner para leer desde teclado.
- Se crea una instancia de **ListaDobleCircular**.
- Se utiliza un ciclo do-while para mostrar el menú repetidamente hasta que el usuario seleccione salir.
- Cada opción del menú llama al método correspondiente de la lista.

UNIVERSIDAD DA VINCI DE GUATEMALA Guatemala C.A.

- Después de insertar o eliminar, se imprime automáticamente el estado actual, tal como lo pide el documento.

Análisis del funcionamiento

La estructura implementada cumple con los requisitos fundamentales de una lista doblemente enlazada circular:

Mantiene enlaces en ambas direcciones.

Conserva la circularidad en todo momento.

Permite inserciones y eliminaciones eficientes en ambos extremos.

Facilita la búsqueda y visualización de elementos.

Presenta una interacción amigable mediante un menú repetitivo.

El uso de head. anterior como referencia al último nodo simplifica varias operaciones y evita recorridos innecesarios.

Conclusiones

La implementación de una Lista Doblemente Enlazada Circular en Java permitió comprender de forma práctica el manejo de referencias entre nodos y la importancia de mantener la integridad estructural de los enlaces. A través de las operaciones desarrolladas, se comprobó que esta estructura ofrece gran flexibilidad para manipular datos dinámicamente.